Vericoding: Verification-Driven Development from Behavioural Features to Machine-Checked Proofs

The axiomander Toolchain

Gavin Mendel-Gleason
Scidonia

July 2026

Abstract

LLM-driven code generation ("vibecoding") trades correctness for velocity: generated code may run, but it leaves no durable specification, no behavioural guarantee, and no machine-checkable evidence that it satisfies its intended purpose. The thesis of this work is that **executable behavioural contracts form a semantic intermediate language from which testing, runtime checking, differential validation, and machine-checked proof obligations can all be derived**. We describe a *vericoding* workflow in which a human-authored specification is the primary lasting artifact and generated code is a replaceable implementation detail. We present a toolchain, axiomander, to operationalise the approach.

The formal backend targets SnakeletExn, a small imperative language in the HeapLang tradition extended with first-class exceptions — crucial for modelling Python's exception-driven control flow. Specifications are stateful: pre/post-conditions, exception arms, and frames are lowered into a Rocq weakest-precondition calculus over a two-cell heap model. The specification comprises Gherkin feature tables, a mathematical contract (pure, boolean-valued predicates over pre/post-state, frames, and exceptions), and an abstract state schema. From this single specification the framework derives executable scenario checks, telemetry assertions, and machine-checked proof obligations in Rocq (Coq) against a hand-verified weakest-precondition calculus.

1 Introduction

A growing fraction of production code is generated by large language models. The resulting programs often "work" in the sense that they execute without crashing, but they leave nothing behind: no specification of intended behaviour, no evidence of correctness, no reproducible conformance regime for the next iteration of the generated code. We call this pattern *vibecoding*: the developer prompts, the model emits, the result runs — and the only long-term artifact is the code itself, with no human-readable statement of what the code is supposed to do.

Vericoding inverts this relationship. In a vericoding workflow, the *specification* is the primary artifact and the code is a replaceable implementation detail. The specification is human-written and human-readable: Gherkin scenarios that domain experts can review, mathematical contracts that state what must hold before and after each operation, and framed state declarations that bound exactly what each operation may read and write. From this single specification every downstream verification activity is derived.

axiomander is a toolchain for vericoding, and is itself battle-tested: the core contract language and theory machinery were extracted from a production citation-checking platform, and the worked examples include a production-stolen invitation flow that exercises insertion frames, multi-table deltas, and time-as-an-argument expiry.

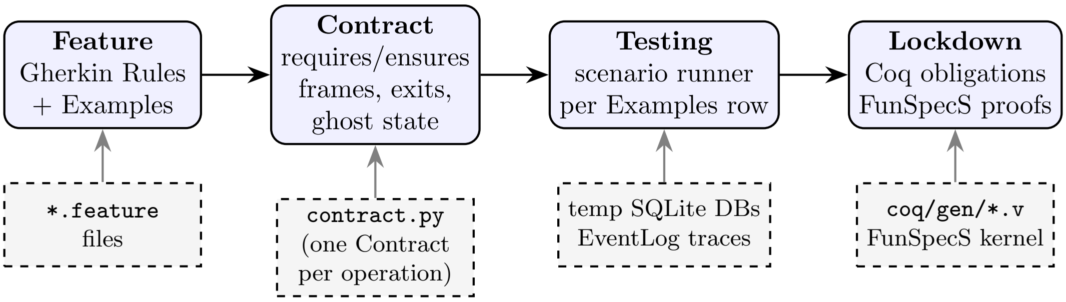

Figure 1: The vericoding pipeline. Features and contracts are the primary lasting artifacts; generated code is a replaceable implementation detail. Every Examples row is exercised against the contract; the same contract lowers to Rocq obligations proved against a hand-verified kernel.

The toolchain supports two development flows. In *contract-first* development, features are written first, then contracts, then an implementation is built against the contract as a specification, then the implementation is tested and lowered to proof. In *retrospective* development, a developer begins with existing feature files and an existing implementation, writes the contract to capture the behaviour already observed in testing, re-runs the test suite under contract checking to validate the specification, and then proceeds to formal proof. Both flows converge on the same long-lived artifact — the contract — and the same endpoint: a machine-checked proof that the specification is internally sound.

The contract language is *fluid* in the Meyer sense: it is a declarative value assembled from named clauses, not imperative assertions scattered throughout the code. More importantly, the same fluid contract serves double duty as an *executable* test criterion and as the *input* to machine-checked proof generation. This is a gradual-typing-like discipline: contracts can be introduced incrementally (testing only), and hardened over time (adding frames, then exceptions, then invariants, then proof obligations), with the toolchain accepting whatever level of specification the domain author is ready to provide.

Figure 1 summarises the pipeline and table 1 walks through a single example step by step.

2 Scientific Contributions

The central thesis of this paper — that a single declarative contract can drive testing, runtime checking, and machine-checked proof — is a realisation of the classical design-by-contract thesis [1, 2], not a new idea in itself. Spec#, Dafny, and JML all pursue variants of this thesis. Our contributions lie in the specific engineering that makes the thesis practical for Python database services, and in three mechanisms with no direct prior art at this granularity.

2.1 Differentially validated library theories

The most distinctive mechanism is the *theory*: a stub for a real library (SQLAlchemy, Python logging, OpenTelemetry) that is given an explicit operational semantics over a pure state model, and that is *differentially validated* against the real library. The same programs are executed against both the stub and the real library, and their observable outputs must agree on the covered fragment; anything outside the fragment is rejected loudly rather than silently approximated. Differential testing is an old technique [14], and adjacent ideas exist in many forms: CompCert's external models for I/O [16], QuickChick's executable semantics for property-based testing [17], K framework's reference interpreters [18], and ordinary mocking frameworks. **We are unaware of previous work combining executable operational semantics for production libraries with differential validation against the concrete implementation.**

It replaces the pervasive but unprincipled use of mocks with a conformance regime that is itself checkable.

2.2 Automatic lowering from Python contracts to Rocq

The lowering pipeline — Python lambdas → AST introspection → shape extraction → Rocq obligation generation over a hand-verified WP calculus — is a novel engineering combination. Why3 extracts to Coq, but from a dedicated specification language [15]; Spec# uses Boogie [2]; Frama-C has its own WP but not Coq back-ends for Python. Our pipeline takes contracts written in a restricted fragment of Python and lowers them into a small imperative language with exceptions (SnakeletExn — HeapLang extended with first-class exceptions, following [10]), emitting FunSpecS-style obligation files with an automatically proven totality lemma, invariant-preservation lemma, and frame-soundness lemma. The dependency-layered emission for parallel proof generation (section 9) is likewise novel.

Foundations in Iris. Our WP calculus is not built from scratch. It is a direct application of the Iris project's `gen_heap` library [9] and follows the weakest-precondition calculus framework of Iris [8]. The two-cell heap model (`store_loc`, `trace_loc`) is a thin layer of FunSpecS-specific specification logic over Iris's standard ghost state and points-to reasoning. All propositional-level proof machinery — lookup/insert lemmas, update application, frame soundness — is standard Iris reasoning, reused without modification. Our contribution is the *lowering*: the mechanical translation from Python contracts into Iris-style specification predicates, not the calculus itself.

2.3 Semantic frame discipline with insertion

Frame conditions are old. Eiffel has explicit modifies clauses [1]; Frama-C has `assigns`; Dafny has `modifies`. Our frame system is an engineering refinement: declarative write paths (keyed-row, keyed-attribute, event-channel) generate the complement automatically, eliminating the "everything else unchanged" boilerplate that dominates classical DbC specifications. The keyed-row *insert* discipline — keyed-row write paths may add the args-resolved key, while deletion is always rejected — is a specific rule we have not seen elsewhere. Event-channel frames, covering structured telemetry as first-class frame-citizens, are likewise distinctive.

2.4 Positioning of other machinery

The symmetric projection — one observation function applied before and after execution — is standard practice in program verification [13], not a contribution; we use it because it is the correct discipline. The declarative `SqlDomain` specification that collapses per-domain infrastructure is an engineering ergonomic, not a scientific contribution. The contract language itself — pure, boolean-valued Python predicates — is a design choice motivated by Python familiarity, not a formal advance.

3 Formal Model

Before describing the implementation we fix the semantic objects the toolchain reasons about. The state of a system is partitioned into three strata.

Definition 1 (Specification state)**.** $\Sigma = \langle \mathcal{O}, \mathcal{D}, \mathcal{G} \rangle$ *where* $\mathcal{O}$ *is the* observed state *(a tuple of typed maps and event-channel tuples read from concrete execution),* $\mathcal{D}$ *is the* derived state *(a tuple of pure computations over* $\mathcal{O}$*, equipped with a consistency check* $\mathsf{derive} : \mathcal{O} \to \mathcal{D}$*), and* $\mathcal{G}$ *is* ghost state *(auxiliary specification-only state invisible to the implementation).*

Definition 2 (Contract satisfaction)**.** *A contract* $C = \langle \Sigma, \mathsf{pre}, \mathsf{post}, \mathcal{X}, \mathcal{I}, \mathcal{W}, \mathcal{R} \rangle$ *is* satisfied *by an execution* $\sigma \xrightarrow{a} \sigma'$ *with result* r *(where* a *are the call arguments) iff:*

(i) $\mathsf{pre}(\sigma, a)$ *holds (admissibility);*

(ii) if no exception condition in $\mathcal{X}$ *holds, then* $\mathsf{post}(\sigma, a, r, \sigma')$ *holds, and every state path outside* $\mathcal{W}$ *is unchanged between* σ *and* σ'*;*

(iii) if an exception condition $C_i \in \mathcal{X}$ *holds, then the raised exception matches* E_i *and every state path outside the exit-specific frame* W_i *is unchanged.*

(iv) $\mathcal{I}(\sigma)$ *and* $\mathcal{I}(\sigma')$ *hold (invariants preserved);*

(v) $\mathsf{derive}(\sigma.\mathcal{O}) = \sigma.\mathcal{D}$ *and* $\mathsf{derive}(\sigma'.\mathcal{O}) = \sigma'.\mathcal{D}$ *(derived consistency).*

Definition 3 (Semantic frame)**.** *Let* $\mathcal{W}$ *be a set of* write paths*: strings of the form* $map[key].attr$*,* $map[key]$*, or* $channel$*. For a write path* $p \in \mathcal{W}$ *whose key is the argument attribute* k*, the frame checker verifies:*

- $\sigma.\mathcal{O}[p] = \sigma'.\mathcal{O}[p]$ *for every keyed path not in* $\mathcal{W}$ *(attribute stability);*
- *the key sets of every keyed map are equal between* σ *and* σ' *except for keyed-row write paths, which may add the args-resolved key (insertion) but never delete.*

Definition 4 (Projection symmetry)**.** *A projection* $\mathsf{snap} : Ctx \to \Sigma$ *is* symmetric *if it is applied identically before and after execution:* $\sigma^{\mathsf{pre}} = \mathsf{snap}(c)$*,* $\sigma^{\mathsf{post}} = \mathsf{snap}(\mathsf{exec}(c))$*, and contract satisfaction is decided over* $(\sigma^{\mathsf{pre}}, a, r, \sigma^{\mathsf{post}})$*.*

Definition 5 (Theory)**.** *A* theory *for a library* L *is a tuple* $\langle \mathcal{A}, \mathcal{E}, S, H, \mathsf{adapt}, \mathsf{val} \rangle$ *where* $\mathcal{A}$ *is the action model (covered fragment of* L*),* $\mathcal{E}$ *is the event model (trace entries),* S *is the pure state model,* H *is a stub handler interpreting actions over* S *and recording a trace,* adapt *connects* H *to* L*'s API so unmodified service code runs against the stub, and* val *is the* validation relation *(a set of proof obligations, one per program in the differential suite, each stating that the stub's observable output equals the real library's output on the covered fragment). The validation relation is established by differential testing; it is the evidence that the theory is sound, not a mere mock.*

4 Contract Language

A axiomander contract is a first-class Python object built by the `Contract` constructor in the *fluid contract* style [1, 2]: the contract is a declarative value assembled from named clauses, not imperative assertions scattered throughout the code.

The contract language is the **pure terminating fragment of Python**: every clause is a lambda whose body may contain only arithmetic, comparison, boolean operators, attribute access, and a small fixed set of combinator calls (`extends_by_one`, `implies`, `safe_div`). Side effects, loops, exceptions, and non-boolean returns are rejected by a static purity checker before the clause is executed or lowered. Every clause must evaluate to a Python `bool`.

Definition 6 (Contract record)**.** *A contract for an operation is* $\langle \Sigma, args, \mathsf{pre}, \mathsf{post}, \mathcal{X}, \mathcal{I}, \mathcal{D}, \mathcal{W}, \mathcal{R}, \mathcal{G} \rangle$ *where:*

- Σ *is the specification state type;*
- *args is a typed argument record (a frozen dataclass);*
- $\mathsf{pre}(\sigma, a)$ *is a boolean predicate (admissibility);*
- $\mathsf{post}(\sigma, a, r, \sigma')$ *is a boolean predicate (success obligations);*
- $\mathcal{X} = \{\, \langle E_i, C_i, W_i, P_i \rangle \mid i \in [1..n] \,\}$ *is the set of exceptional exits;*
- $\mathcal{I}$ *is the invariant set;* $\mathcal{D}$ *maps derived field names to pure functions;* $\mathcal{W}$ *and* $\mathcal{R}$ *are the write and read frame path sets;* $\mathcal{G}$ *carries ghost type, initialisation, transitions, and invariants.*

4.1 Semantic frames

Frame conditions in classical DbC are notoriously verbose. In axiomander the contract declares only what it *may* write:

$$\mathcal{W} = \{\ \text{“state.products[sku].reserved”},\ \text{“state.invitations[token]”},\ \text{“state.reservation\_log”}\ \}$$

The checker derives frame soundness automatically: every attribute outside $\mathcal{W}$ must be equal, every key outside the args-resolved write paths must have stable key sets, and deletions are always rejected. Keyed-row paths may *add* the args-resolved key — essential for operations such as `invite` that create rows.

4.2 Exceptional exits

Exceptions are specified as data:

$$\mathcal{X} = \left\{\ \langle \mathsf{InsufficientStock},\ s.available(a.sku) < a.quantity,\ \{failure\_log\},\ \mathsf{failure\_ensures}\rangle\ \right\}$$

The runner checks that when the `when` condition holds, the correct exception is raised, the exit's frame is respected, and the failure telemetry contains exactly one new event with the declared fields. The classical “exceptions leave state untouched” discipline is the default empty frame.

4.3 Telemetry: `extends_by_one`

The combinator $\mathsf{extends\_by\_one}(old, new, pred)$ states that *new* is exactly *old* with one element appended, and the appended element satisfies *pred*. It is the contract language's primitive for asserting event-channel content, used in both success and exit post-conditions. The runtime interpretation is $|new| = |old| + 1 \wedge new[:-1] = old \wedge pred(new[-1])$; the lowering translates it to a list-append obligation in the generated Rocq.

5 Symmetric Projection

The bridge between the concrete execution world and the abstract specification is a single projection function applied twice.

Given a witness w (from a Gherkin Examples row), the materializer $M(w)$ produces an execution context: a temp SQLite file, an engine pointing at it, and a fresh event log. The scenario runner then executes a fixed six-stage check per row: materialize, pre-check (invariants + admissibility), execute (service + effect-emitting wrapper), frame-check, derived-check, post-check (ensures + invariants).

Crucially, the service is ordinary production code: SQLAlchemy `engine.begin()` transactions, `text()` statements, domain exceptions. Nothing in it knows about contracts, snapshots, or scenario rows.

6 Examples with Databases

We present three domains of increasing structural interest, all against SQLite through SQLAlchemy. Table 2 summarises their shapes and table 3 gives quantitative measures.

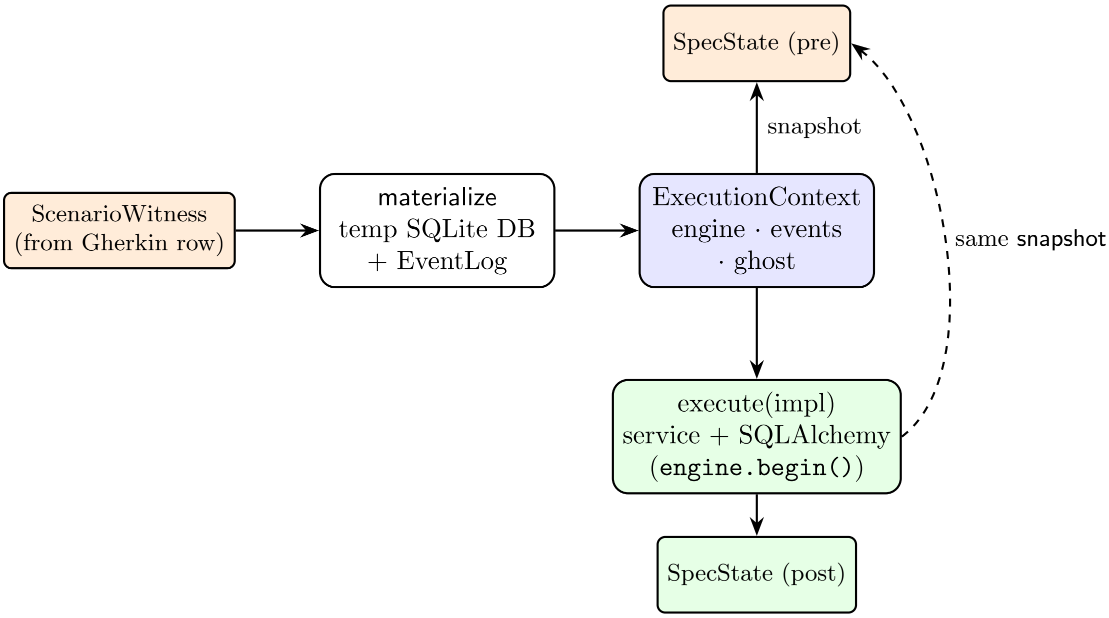

Figure 2: The symmetric projection. snapshot(context) projects the concrete world both before and after execution; contracts compare pre and post. The service is ordinary production code against SQLAlchemy; the same code runs on a real SQLite file or the theory stub (section 7).

6.1 Inventory: deltas and conditional telemetry

The inventory domain models stock reservations over a single `products` table. Contracts for `reserve`, `release`, and `restock` share the same state schema, invariants ($0 \le \mathit{reserved} \le \mathit{on\_hand}$ for every product), and derived computations (total on-hand, reserved, available).

$$\mathsf{pre}(s, a) \;=\; a.\mathit{quantity} > 0 \;\wedge\; a.\mathit{sku} \in s.\mathit{products}$$

$$\begin{aligned}
\mathsf{post}(s, a, r, s') \;=\;\; & s'.\mathit{products}[a.\mathit{sku}].\mathit{reserved} \;=\; s[\,\cdot\,].\mathit{reserved} + a.\mathit{quantity} \\
& \wedge\; \mathtt{e1}(s.\mathit{reservation_log},\, s'.\mathit{reservation_log},\, \lambda e.\ e.\mathit{sku} = a.\mathit{sku}) \\
& \wedge\; \mathtt{e1}(s.\mathit{gauge_log},\, s'.\mathit{gauge_log},\, \lambda g.\ g.\mathit{sku} = a.\mathit{sku} \wedge g.\mathit{on_hand} = s'.\mathit{on_hand}) \\
& \wedge\; \mathsf{implies}(\mathit{crossed}(s, a, s'),\, \mathtt{e1}(s.\mathit{alert_log},\, s'.\mathit{alert_log},\, \ldots))
\end{aligned}$$

The alert condition is edge-triggered: available stock crossed from above the reorder point to at-or-below it.

6.2 Bank transfer: multi-key deltas and invariants

The transfer contract moves funds between two rows of one table, with a balance-nonnegativity invariant and currency-mismatch and insufficient-funds exits. Its writes span two different keyed attributes on different keys, demonstrating that frames compose over multi-delta operations.

6.3 Invitations: insertion, multi-table state, and time

The invitations domain exercises the newest machinery: insert frames, multi-table acceptance, and time-as-an-argument expiry.

$$\begin{aligned}
\mathsf{post}_{\mathit{invite}}(s, a, r, s') \;=\;\; & s'.\mathit{invitations}[a.\mathit{token}].\mathit{status} = \text{“pending”} \\
& \wedge\; s'.\mathit{invitations}[a.\mathit{token}].\mathit{expires_at} = a.\mathit{now} + 7{\cdot}86400
\end{aligned}$$

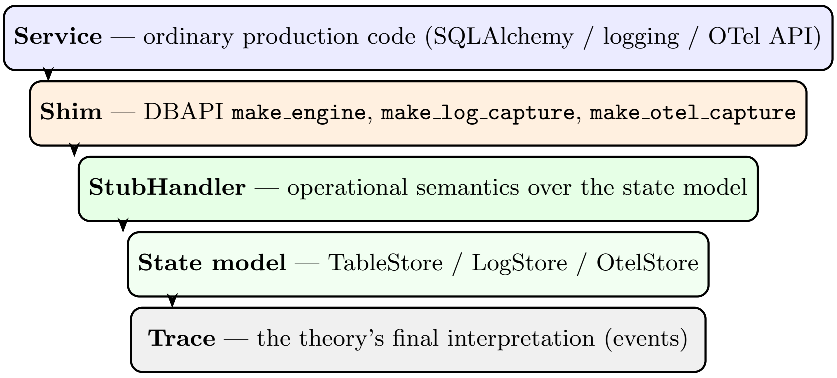

Figure 3: The theory stack. The service is written once, against the real library API; the shim connects to the stub; the stub interprets actions over a pure state model and records the trace. Differential tests run the same programs against stub and real library and require identical observable behaviour.

with $\mathcal{W}_{invite} = \{\, invitations[token],\ invite\_log \,\}$ — an insert-frame operation.

$$
\begin{aligned}
\mathsf{post}_{accept}(s, a, r, s') \;=\;\; & s'.invitations[a.token].status = \text{“accepted”} \\
& \wedge\; s'.members[a.user\_id].org\_id = s.invitations[a.token].org\_id \\
& \wedge\; s'.users[a.user\_id].default\_org = s.invitations[a.token].org\_id
\end{aligned}
$$

with $\mathcal{X}_{accept}$ containing $\mathsf{InvitationExpired}$ (time check) and $\mathsf{EmailMismatchError}$ (email mismatch) exits. Token is a caller-supplied argument so the insert key is args-resolved; expiry time is an integer epoch so the comparison is pure.

7 Theories: Library Semantics as Tested Models

A *theory* for a library L is a tuple $\langle \mathcal{A}, \mathcal{E}, S, H, \mathsf{adapt}, \mathsf{val} \rangle$ as defined in section 3. The key property is that the stub handler is validated by *differential testing*: the same program runs against the real library L and against the stub, and their observable outputs must agree on the covered fragment.

7.1 SQL theory

The SQL theory covers a deliberately small fragment: equality-WHERE `SELECT` (with ascending `ORDER BY`), single-row `INSERT`, and `UPDATE` with `SET col = col` $\pm$ `expr`. Statements are parsed by `sqlglot`; anything outside the fragment raises `UnsupportedStatementError` — no silent degradation. The stub handler interprets the action model over an immutable `TableStore`, records the trace, and enforces transaction discipline. A PEP-249 DBAPI shim exposes the handler so *unmodified SQLAlchemy service code runs against the theory*. The sqlite dialect's deferred-autobegin semantics is modelled at the connection level; `commit()` and `rollback()` are, as in real sqlite3, no-ops without an open transaction.

7.2 Logging and OpenTelemetry

The same six-layer shape repeats for logging and OpenTelemetry theories. Both ship validation suites that compare stub output against the real library. The logging theory captures every `logger.info/warning/error` call as a structured `LogEmit` record; the OTel theory captures the full span lifecycle as a `SpanRecord`.

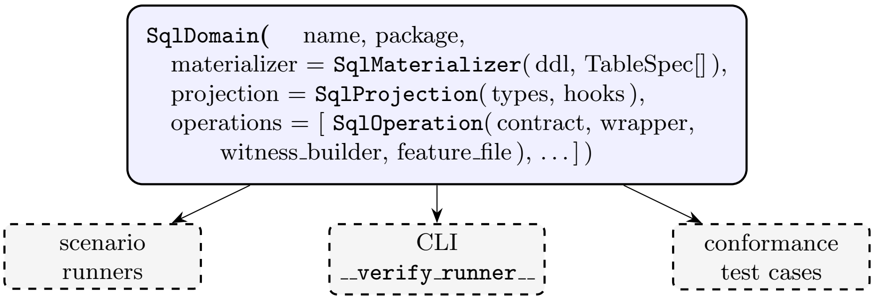

Figure 4: One `SqlDomain` declaration derives runners, CLI dispatch, and the conformance suite.

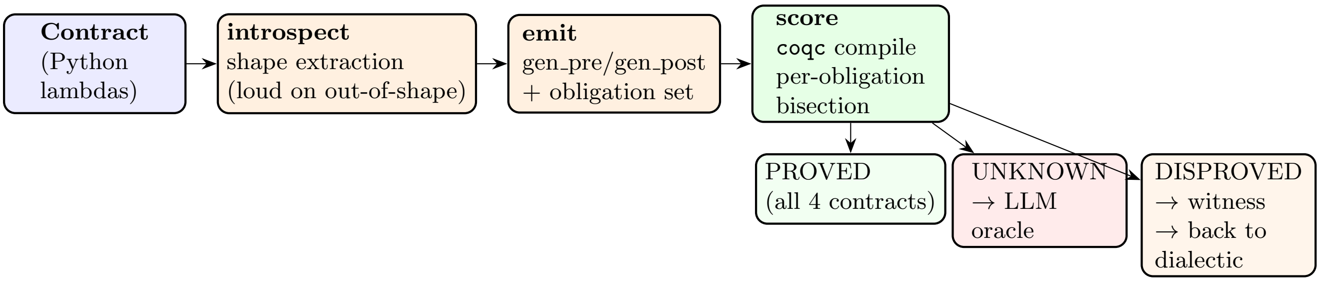

Figure 5: The lowering pipeline: introspect, emit, score. The scoreboard has three outcomes: PROVED (machine-checked), UNKNOWN (queued for the proof oracle), and DISPROVED (a runtime counterexample is surfaced — a concrete witness of specification failure).

Principle 1 (Theory conformance)**.** *A theory is admissible only if: (i) its stub and the real library agree observationally on a differential suite; (ii) its translator rejects out-of-fragment input loudly; (iii) its state model and trace are pure data. New theories follow the same checklist.*

8 Domain Declarations

Experience with three domains revealed a stable shape: only types, service, contracts, features, and witness builders are genuinely domain content; everything else was byte-identical infrastructure copied per domain. The `SqlDomain` declaration collapses the latter.

A `TableSpec` names a table's columns, primary key, and the witness attribute carrying its initial rows; from these the materializer generates DDL execution, `DELETE + INSERT` seeding, engine construction, and teardown. The projection queries each declared table and passes row dicts plus the event log to two small domain hooks. Domains with irregular tables declare a `populate_extra` hook.

9 Lowering Architecture

The lowering pipeline transforms a Python `Contract` into a Rocq obligation file and scores each obligation against a hand-verified kernel. The pipeline has three stages: introspect, emit, and score.

9.1 The Rocq kernel

The Coq side comprises three layers. **SnakeletExnLang** is a small imperative language in the HeapLang tradition of the Iris project, extended with first-class exceptions (following van Collem, de Vilhena and Krebbers) — crucial for modelling Python's exception-driven control flow. **SnakeletExnWp** provides a weakest-precondition calculus over it with stateful opaque specifications in the FunSpecS style (pure, stateful opaque, and transparent function entries). **SpecPrelude** provides dictionary lookup/insert helpers and their correctness lemmas, proven once rather than per contract.

The specification kernel (**FunSpecS**) maps operation names to stateful specs $(pre, post)$ where $post(\sigma, vs, r, ups)$ constrains the result $r \in \{\, \mathsf{RVal}(v), \mathsf{RExn}(\ell, p) \,\}$ and the list of heap-cell updates ups. The kernel guarantees that each spec is total: for every pre-state σ and argument list vs satisfying $pre(\sigma, vs)$, there exist a result r and updates ups satisfying $post(\sigma, vs, r, ups)$ and respecting the map domain.

9.2 Shape extraction (introspect)

The introspector (`introspect.py`) parses each clause's Python AST into a `ContractInfo` shape object:

- **Deltas**: expressions of the form $new.map[key].field = old.map[key].field \pm args.qty$ are extracted as $(key, field, op, qty)$ tuples.
- **Scalars**: simple comparisons ($args.x > 0$, $args.x \neq args.y$) are reduced to (lhs, op, rhs) triples.
- **Exceptions**: the class name, conjunctive when-conditions, and payload fields are extracted.
- **Traces**: `extends_by_one` calls are detected and deferred to a later lowering pass.
- **Out-of-shape rejection**: anything not matching the supported fragment raises `UnsupportedShapeError` Every clause is gate-checked for purity.

9.3 Emission

The emitter (`emit.py`) translates a `ContractInfo` into a complete Rocq file. The generated structure per operation is:

- **Heap model**: two locations, *store_loc* (the table dict) and *trace_loc* (the event dict).
- **Row constructors**: $\mathsf{row\_of}(f_1, \ldots, f_k)$ builds a `LitDict` with typed fields.
- **Row invariant**: $\mathsf{row\_inv}(v)$ holds iff there exist fields such that $v = \mathsf{row\_of}(\ldots)$ and the invariant conjuncts hold (e.g. non-negativity, ordering).
- **Store invariant**: $\mathsf{store\_inv}(\mathsf{kvs})$ lifts $\mathsf{row\_inv}$ over all entries.
- **Pre/post**: gen_pre and gen_post bind all free variables existentially, look up each key's row via dict_lookup_str, conjunct the scalar preconditions, and (for post) describe the result and the *nested insert* of all deltas.
- **Function table**: gen_table maps operation names to FunSpecS entries, with one arm per exception exit.
- **Totality lemmas**: gen_table_total and gen_table_total_pure prove that the table satisfies the FunCtx requirements.
- **Per-delta preservation**: each delta gets a lemma showing the store invariant survives the single-row update.

```
Definition gen_post (sigma : sn_state) (vs : list sn_val)
    (r : Result) (ups : cell_updates) : Prop :=
  exists sku order quantity store_d on_hand_sku
         reserved_sku reorder_point_sku,
    vs = [LitString sku; LitString order; LitInt quantity] /\
    sigma !! store_loc = Some (LitDict store_d) /\
    dict_lookup_str sku store_d =
      Some (row_of on_hand_sku reserved_sku reorder_point_sku) /\
    r = RVal (LitInt reserved_sku) /\
    ups = [(store_loc, LitDict (dict_insert_str sku
            (row_of on_hand_sku (reserved_sku + quantity)
                    reorder_point_sku) store_d))].
```

9.4 Obligation set

The generated file contains eight numbered obligations:

Obligation	Statement
O1	Admissibility sanity: $\exists \sigma, vs.\ \mathsf{gen\_pre}(\sigma, vs) \wedge \mathsf{store\_inv}(\sigma)$ — the spec is non-vacuous.
O2	Spec consistency: $\mathsf{gen\_pre}(\sigma, vs) \implies \exists r, ups.\ \mathsf{gen\_post}(\sigma, vs, r, ups) \wedge \mathsf{updates\_dom\_in}(\sigma, ups)$ — the success path is reachable.
O3.i	Each exception arm is satisfiable: given its pre, there exists a result and updates satisfying its post.
O4	Exit coverage: the success availability and the disjunction of all exit when-conditions partition the state space.
O5	Invariant preservation across all nested delta updates.
O8	Frame soundness: for any location $l \neq \mathsf{store\_loc}$, $\mathsf{apply\_updates}(\sigma, ups)!!l = \sigma!!l$.

9.5 Proof harness

`harness.py` first compiles the whole file with `coqc`. If the whole-file compile succeeds, every obligation is labelled PROVED. On failure, it *bisects*: for each obligation lemma, it generates a variant file where every other lemma is replaced with `Proof. Admitted.`, then compiles the variant in isolation. Obligations that compile in isolation are PROVED; the remainder are UNKNOWN, packaged as self-contained proof problems for an LLM-based oracle.

A third status, DISPROVED, is the natural extension when a runtime check fails: the scenario runner already exercises every contract clause against concrete data, and a counterexample for a well-formed contract means the specification is inconsistent.

10 Trusted Computing Base

The following components are **trusted** (not themselves machine-proved):

- The Rocq kernel and the `coqc` proof checker.
- The `FunSpecS` kernel and `SnakeletExnWp` calculus: hand-written Rocq that must correctly model the intended operational semantics.
- The `SpecPrelude` dictionary helpers.
- The lowering implementation itself: `introspect.py` (AST extraction and shape recognition), `emit.py` (translation to Rocq), `frames.py` (frame checking), and `purity.py` (static purity validation).

The following are **proved** by the generated obligations: admissibility non-vacuity, post-state consistency, exit coverage, invariant preservation, and frame soundness – relative to the kernel. The following are **not yet proved**: refinement between the Coq model and the concrete Python implementation (the scenario runner tests this empirically); trace obligations in the lowering; and the correctness of the emitter itself.

11 Evaluation

Table 3 gives quantitative measures. All 23 store obligations across four contracts are fully proved against the Rocq kernel. The lowering additionally emits obligations in dependency layers (one file per layer, plus a `schedule.json` DAG) for parallel proof orchestration by rocqpiler; each layer compiles independently. Trace obligations (list-append post-conditions for event channels) are enforced at runtime by the scenario runner and are the active development front in the lowering.

The three example domains exercise a range of verification scenarios: single-table keyed deltas (inventory), multi-key deltas with a conservation invariant (bank transfer), and insert-frame multi-table deltas with time-based exception exits (invitations). All 42 feature rows are exercised by the generic conformance suite.

11.1 Implemented versus future work

The following are **implemented** and operational: executable contracts with pre/post conditions and exception exits; semantic frame checking; runtime scenario verification; domain declarations with generic conformance testing; the SQL, logging, and OpenTelemetry theories with validation suites; the lowering pipeline with shape extraction, emission, and proof scoring; and all 23 store obligations machine-proved.

The following are **future work**: trace-cell obligations in the lowering; user-supplied ghost algebras; incremental re-verification keyed on contract/body hashes; loop and sequence support in the obligation generator; DISPROVED counterexample surfacing; and expanding the supported language fragment.

12 Related Work

- **Design by Contract** (Meyer [1], Eiffel): provides runtime assertions without a path to mechanised proof. axiomander keeps the executable-contract ergonomics and adds the lowering to Rocq, plus frame inference that eliminates the verbosity of manual frame conditions.
- **Spec#** (Leino and Müller [2]): a language-integrated contract system with static verification via Boogie. Like Spec#, axiomander treats contracts as first-class; unlike Spec#, the contracts are not tied to the implementation language and persist independently of the code.
- **Dafny, Frama-C, KeY, Why3**: verify source-level contracts but require the source language to be the specification language. axiomander instead verifies an abstract model aligned with the declared frame, keeping production code unmodified and language-agnostic.
- **Iris separation logic** (Jung et al.): provides a framework for building WP calculi. axiomander uses the Iris `gen_heap` library as infrastructure for its Rocq kernel and shares the heap-cell update model.
- **Property-based testing** (QuickCheck [5], Hypothesis): generates random inputs and checks properties. axiomander contracts are simultaneously the property oracle and the proof input; the Gherkin tables provide the concrete test data.
- **Behaviour-driven development** (Gherkin [4]): specifies behaviour in scenario tables. axiomander connects Gherkin to executable contracts and formal proof, so the scenarios become verified test data.
- **TLA$^+$**: specifies systems in temporal logic. axiomander targets operational semantics and explicit state transitions with a considerably lower barrier to entry (Python lambdas vs. TLA$^+$ formulas).

Internal validity. The lowering pipeline is trusted code: the emitter's translation from `ContractInfo` to Rocq is not itself proved correct. The frame checker and purity validator are similarly trusted. The semantic object of the proof is the contract, not the implementation — the scenario runner tests the latter empirically.

External validity. The three example domains are small but representative of database-backed services: they exercise single-table, multi-table, insert-bearing, and time-bearing patterns. The theory mechanism has been demonstrated for three libraries (SQL, logging, OTel) but the approach to adding new theories is manual.

Construct validity. The claim that contracts form a "semantic intermediate language" is currently instantiated via a specific AST extraction and lowering pipeline. The contract fragment is restricted to what the introspector and emitter can handle; expanding the fragment is future work.

13 Conclusion

This paper has presented the thesis that executable behavioural contracts form a semantic intermediate language from which testing, runtime checking, differential validation, and machine-checked proof obligations can all be derived. A vericoding workflow leaves the specification as the lasting artifact and the code as a replaceable implementation detail.

The axiomander toolchain operationalises this thesis through six contributions: contracts as semantic intermediate representation, semantic frame inference, differentially validated library theories, a symmetric projection architecture, automatic lowering to Rocq proof obligations, and declarative domain specification. The implementation is not a proposal — it exists, it works, and the evidence is machine-checked: 23 store obligations across four contracts are proved against a hand-verified WP calculus.

References

[1] Bertrand Meyer. *Applying "Design by Contract"*. IEEE Computer, 25(10):40–51, 1992.

[2] K. Rustan M. Leino and Peter Müller. *Using the Spec# Language, Methodology, and Tools to Write Bug-Free Programs.* In LASER Summer School 2008/2009, LNCS 6029, Springer, 2010.

[3] Gavin Mendel-Gleason. *axiomander: Specification-Driven Verification.* Source code and documentation. `https://github.com/scidonia/axiomander`, 2025–2026.

[4] Cucumber Ltd. *Gherkin Reference.* `https://cucumber.io/docs/gherkin/reference/`, 2025.

[5] Koen Claessen and John Hughes. *QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs.* In ICFP 2000.

[6] John C. Reynolds. *Separation Logic: A Logic for Shared Mutable Data Structures.* In LICS 2002.

[7] Scidonia. *SnakeletExn and FunSpecS: A Small Imperative Language and Weakest-Precondition Specification Kernel.* Coq development, 2026.

[8] Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, et al. *Iris: Monoids and Invariants as an Orthogonal Basis for Concurrent Reasoning.* In POPL 2018.

[9] Ralf Jung. *The Iris Project.* `https://iris-project.org/`, 2018.

[10] Tom Van Collem, Bas de Vilhena, and Robbert Krebbers. *Exceptions in the Snakelet Language.* Coq development, 2026.

[11] The Iris Project. *HeapLang: A Simple Imperative Language for Verification.* `https://iris-project.org/`, 2026.

[12] Jeremy Tobin-Hochstadt and Matthias Felleisen. *The Design and Implementation of Typed Scheme*. In POPL 2008.

[13] Patrick Cousot and Radhia Cousot. *Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints*. In POPL 1977.

[14] William McKeeman. *Differential Testing for Software*. Digital Technical Journal, 10(1):100–107, 1998.

[15] Jean-Christophe Filliâtre. *Why3: A Platform for Deductive Program Verification*. In CAV 2013.

[16] Xavier Leroy et al. *CompCert: Formally Verified Optimizing Compiler*. `https://compcert.org/`, 2016.

[17] Zoe Paraskevopoulou et al. *QuickChick: Property-Based Testing in Coq*. JFP, 2022.

[18] Grigore Roşu. *From Rewriting Logic, to Programming Language Semantics, to Program Verification*. In LOPSTR 2013.

Stage	**What happens**
1. Feature	A Gherkin `reserve.feature` contains: `Given a product "S1" with on-hand 100 reserved 10 ...` with an Examples table of concrete rows.
2. Witness	A pure Python function maps each row to a typed `ReserveArgs(sku="S1", order_id="O1", quantity=30)` and an initial products dict: {`"S1": Product(on_hand=100, reserved=10, ...)`}.
3. Materialize	A temp SQLite file is created, DDL run, and rows inserted. The concrete world is: `ExecutionContext(engine=create_engine("sqlite:///tmp/..."), events=EventLog())`.
4. Pre-snapshot	`snapshot(context)` → `SpecState`: reads `products` via SQLAlchemy, partitions the empty event log, computes derived totals, and returns an immutable frozen dataclass.
5. Admissibility	`requires(state, args)` is evaluated: `args.quantity > 0` ∧ `args.sku` ∈ `state.observed.products`. If `outcome = "rejected"`, admissibility is expected to fail; for `"success"`, it must hold.
6. Invoke service	`InventoryService().reserve(engine, "S1", "O1", 30)` runs against the real SQLite file through SQLAlchemy. On success it returns a `ReservationReceipt`; the effect-emitting wrapper records `StockReserved` and gauge events in the context's event log.
7. Post-snapshot	The **same** `snapshot` function runs against the now-mutated SQLite file, producing the post-state `SpecState` with updated reserved count and appended event tuples.
8. Contract check	The runner evaluates: (a) **frame** — everything outside `writes` unchanged; (b) **derives** — recomputed aggregates consistent; (c) **ensures** — reserved increased by exactly the quantity, exactly one `StockReserved` event emitted with exact fields; (d) **invariants** — reserved ≤ on-hand, both non-negative.
9. Lower	The same `Contract` object is introspected by `axiomander.lower`: deltas, scalars, and exception arms are extracted; a Rocq obligation file is emitted. `coqc` compiles it; per-obligation bisection yields a scoreboard.
10. Prove	`coqc -R coq "" coq/gen/GenReserveObligations.v` succeeds. O1–O8 are machine-checked: admissibility sanity, spec consistency, exit coverage, invariant preservation, and frame soundness — all PROVED.

Table 1: End-to-end walkthrough of the reserve contract. Ten stages, one spec, three levels of assurance.

Domain	Tables	Operations	Feature rows	Obligations
inventory	1	reserve, release, restock	22	$6/6 \cdot 6/6 \cdot 4/4$
bank_transfer	2	transfer	11	7/7
invitations	3	invite, accept	9	(runtime verification)

Table 2: The three database-backed example domains. All feature rows are exercised by the conformance suite; 23 store obligations across 4 contracts are machine-proved.

Metric	Value
Python source lines (axiomander core + examples)	$\approx$ 10,000
Rocq source lines (kernel + proofs)	$\approx$ 3,500
Number of example domains	3
Number of contracts	6
Number of feature rows exercised	42
Generated obligations per contract	4–8
Store obligations proved	23/23
Trace obligations (runtime only)	deferred
Runtime verification per row	< 0.1 s
Lowering time (introspect + emit)	< 0.5 s per contract
Rocq compilation per obligation file	< 30 s

Table 3: Quantitative measures of the axiomander implementation. All store obligations are machine-proved.